Générateurs de Nombres Aléatoires

Justification Complète : Théorie, Implémentation et Attaques

ICE 3A – Projet RNG

O.Denis

Janvier – Février 2026

Note de responsabilité

Les attaques présentées dans ce document sont strictement pédagogiques. Elles visent à illustrer les faiblesses théoriques de certains algorithmes dans un cadre académique contrôlé. Toute reproduction de ces techniques sur des systèmes réels sans autorisation explicite est illégale et passible de poursuites judiciaires au titre de la loi Godfrain (art. 323-1 et suivants du Code pénal). Les auteurs déclinent toute responsabilité quant à une utilisation malveillante du contenu de ce rapport.

Table des matières

1 Introduction **4**
1.1 Taxonomie des générateurs 4
1.2 Propriétés d'un CSPRNG 4

2 Générateurs **5**
2.1 Linear Congruential Generator (LCG) 5
2.1.1 Théorie mathématique 5
2.1.2 Paramètres standards 6
2.1.3 Justification du code 6
2.1.4 Interprétation et application 7
2.2 Mersenne Twister (MT19937) 7
2.2.1 Théorie mathématique 7
2.2.2 Justification du code 8
2.2.3 Interprétation et application 10
2.3 Transformée de Box-Muller 10
2.3.1 Théorie mathématique 10
2.3.2 Justification du code 11
2.4 Blum-Blum-Shub (BBS) 12
2.4.1 Théorie mathématique 12
2.4.2 Justification du code 13
2.5 Hash_DRBG (NIST SP 800-90A) 14
2.5.1 Théorie mathématique 14
2.5.2 Justification du code 15
2.6 os.urandom 17
2.6.1 Théorie : sources d'entropie physique 17
2.6.2 Justification du code 18
2.7 XOR NRBG (Non-Deterministic Random Bit Generator) 18
2.7.1 Théorie mathématique 18
2.7.2 Justification du code 19

3 Tests statistiques **21**
3.1 Entropie de Shannon . . . 21
3.1.1 Théorie mathématique . . . 21
3.1.2 Justification du code . . . 22
3.2 Test du Chi-carré . . . 22
3.2.1 Théorie mathématique . . . 22
3.2.2 Justification du code . . . 23
3.3 Autocorrélation . . . 24
3.3.1 Théorie mathématique . . . 24
3.3.2 Justification du code . . . 24
3.4 Test de Kolmogorov-Smirnov (KS) . . . 25
3.4.1 Théorie mathématique . . . 25
3.4.2 Justification du code . . . 25

4 Attaques **27**
4.1 Récupération de la graine LCG . . . 27
4.1.1 Modèle de menace . . . 27
4.1.2 Méthode 1 : Résolution algébrique . . . 27
4.1.3 Méthode 2 : Force brute . . . 28
4.1.4 Méthode 3 : Known-plaintext (chiffrement XOR) . . . 28
4.2 Reconstruction d'état du Mersenne Twister . . . 29
4.2.1 Modèle de menace . . . 29
4.2.2 Inversibilité du tempering . . . 29
4.2.3 Reconstruction et prédiction . . . 30

5 Recommandations et analyse critique **32**
5.1 Tests statistiques $\neq$ sécurité cryptographique . . . 32
5.2 Principe de Kerckhoffs . . . 32
5.3 Incidents réels . . . 32
5.4 Tableau comparatif des générateurs . . . 33
5.5 Règles fondamentales . . . 33

6 Conclusion **34**

Bibliographie **35**

Chapitre 1

Introduction

Ce document constitue la justification complète et détaillée de chaque composant du projet. Pour chaque générateur, test statistique et attaque, nous fournissons : la théorie mathématique sous-jacente avec preuves formelles, la justification du code ligne par ligne, l'interprétation des résultats et les applications pratiques.

1.1 Taxonomie des générateurs

Les générateurs de nombres aléatoires se divisent en trois grandes familles :

Table 1.1 – Classification des générateurs de nombres aléatoires

Famille	**Exemples**	**Propriétés**
PRNG non-crypto	LCG, MT19937	Rapides, pas de garantie cryptographique
CSPRNG	BBS, Hash_DRBG	Imprévisibilité prouvée
NRBG / Hybrides	os.urandom, XOR NRBG	Entropie physique réelle

1.2 Propriétés d'un CSPRNG

Un générateur cryptographiquement sûr (CSPRNG) doit satisfaire deux propriétés absentes des PRNG classiques :

1. **Next-bit unpredictability** : connaître les k premiers bits ne permet pas de prédire le $(k+1)$-ième avec probabilité supérieure à $\frac{1}{2} + \varepsilon$ pour tout ε négligeable.
2. **State compromise extension** : compromettre l'état courant ne permet pas de reconstruire les sorties passées.

Chapitre 2

Générateurs

2.1 Linear Congruential Generator (LCG)

2.1.1 Théorie mathématique

Le LCG repose sur la **récurrence linéaire modulaire** :

$$X_{n+1} = (a \cdot X_n + c) \bmod m \tag{2.1}$$

où X_0 est la *graine* (seed), a le multiplicateur, c l'incrément, et m le module.

Théorème 2.1.1 (Hull-Dobell – condition de pleine période)**.** *Un LCG atteint la période maximale m si et seulement si les trois conditions suivantes sont simultanément satisfaites :*

1. $\gcd(c, m) = 1$ *(c et m sont premiers entre eux) ;*
2. $(a - 1)$ *est divisible par tous les facteurs premiers de m ;*
3. *Si $4 \mid m$, alors $4 \mid (a - 1)$.*

Démonstration. La suite (X_n) est périodique car elle évolue dans l'ensemble fini $\{0, \ldots, m-1\}$. La période est maximale si et seulement si $f(x) = (ax+c) \bmod m$ est une permutation de $\mathbb{Z}/m\mathbb{Z}$. Cette fonction est bijective si et seulement si elle est injective : $f(x) \equiv f(y) \pmod m$ implique $a(x - y) \equiv 0 \pmod m$. Pour que ceci entraîne $x \equiv y \pmod m$, il faut $\gcd(a, m) = 1$. Les conditions de Hull-Dobell raffinent ce résultat pour les modules puissances de 2. □

Théorème 2.1.2 (Marsaglia, 1968 – Structure hyperplane)**.** *Pour tout LCG de module m et dimension d, les vecteurs $(X_n, X_{n+1}, \ldots, X_{n+d-1})$ se situent sur au plus $(d! \cdot m)^{1/d}$ hyperplans parallèles dans $[0, 1]^d$.*

Ce théorème implique que les LCG ne peuvent jamais être utilisés en simulation haute-dimensionnelle ni en cryptographie. Le cas pathologique de RANDU illustre ce problème :

avec $a = 65539 = 2^{16} + 3$, on peut montrer que :

$$X_{n+2} = 6\,X_{n+1} - 9\,X_n \pmod{2^{31}}$$

de sorte que les triplets (X_n, X_{n+1}, X_{n+2}) ne se situent que sur **15 plans** dans l'espace 3D.

2.1.2 Paramètres standards

TABLE 2.1 – Paramètres LCG courants

Nom	a	c	m	Qualité
glibc	1 103 515 245	12 345	2^{31}	Passable
RANDU	65 539	0	2^{31}	**Mauvais**
MMIX (Knuth)	6 364 136 223 846 793 005	1 442 695 040 888 963 407	2^{64}	Bon

2.1.3 Justification du code

```python
def lcg(seed, a, c, m, n):
    x = seed            # X_0 : valeur initiale (graine) -- etat interne
    results = []
    for _ in range(n):
        x = (a * x + c) % m    # Recurrence : X_{i+1} = (a*X_i + c) mod m
        results.append(x)      # X_0 n'est jamais inclus dans la sortie
    return results              # Retourne n entiers dans [0, m-1]

PARAMS_GLIBC = {'a': 1103515245, 'c': 12345,  'm': 2**31}
PARAMS_RANDU = {'a': 65539,       'c': 0,      'm': 2**31}  # A eviter
PARAMS_KNUTH = {'a': 6364136223846793005,
                'c': 1442695040888963407, 'm': 2**64}
```

Listing 2.1 – Implémentation LCG (generators/lcg.py)

Analyse ligne par ligne :

— `x = seed` : initialise l'état. La séquence est entièrement déterminée par (X_0, a, c, m) ; deux exécutions avec la même graine produisent la même séquence.

— `x = (a*x + c) % m` : applique la récurrence (2.1). En Python, l'arithmétique est en précision arbitraire – aucun dépassement.

— `results.append(x)` : stocke X_{i+1} (la graine X_0 n'est jamais exposée).
— Les paramètres RANDU sont inclus à titre pédagogique pour démontrer les conséquences d'un mauvais choix.

2.1.4 Interprétation et application

Applications légitimes : simulation physique, jeux vidéo, algorithmes randomisés non critiques, tests unitaires reproductibles.

Applications interdites : génération de clés, nonces, IV, tokens d'authentification – toute sortie accessible à un attaquant.

2.2 Mersenne Twister (MT19937)

2.2.1 Théorie mathématique

Le MT19937 (Matsumoto & Nishimura, 1998) est basé sur la théorie des **corps de Galois** GF(2). L'état interne est un vecteur de $N = 624$ mots de 32 bits (état total de 19 937 bits).

Récurrence sur GF(2)

La récurrence s'écrit :

$$x_{k+n} = x_{k+m} \oplus \big((x_k^u \,|\, x_{k+1}^l) \cdot A\big) \tag{2.2}$$

où x_k^u désigne le bit de poids fort de x_k, x_{k+1}^l les 31 bits de poids faible de x_{k+1}, et A est la matrice de twist définie par :

$$A \cdot x = \begin{cases} x \gg 1 & \text{si LSB}(x) = 0 \\ (x \gg 1) \oplus \texttt{0x9908B0DF} & \text{si LSB}(x) = 1 \end{cases}$$

Théorème 2.2.1 (Période du MT19937)**.** *La période de la suite générée par la récurrence (2.2) est* $2^{19937} - 1$.

Sketch de preuve. La récurrence peut s'exprimer matriciellement sur GF(2) : l'état $(x_0, \ldots, x_{n-1})$ est multiplié par une matrice de transition T de taille 19937×19937 sur GF(2). La période est $2^{19937} - 1$ car le polynôme caractéristique de T est **primitif** sur GF(2), de degré 19937. Un polynôme primitif de degré p génère un cycle de longueur $2^p - 1$ dans $\mathrm{GF}(2)^p \setminus \{0\}$. Le nombre 19937 est premier, et il a été vérifié que le polynôme associé est bien primitif. □

k-distribution

Théorème 2.2.2 (k-distribution du MT19937)**.** *Le MT19937 est k-distribué à 32 bits de précision pour tout $1 \leq k \leq 623$. Formellement, si l'on considère les vecteurs consécutifs de k sorties normalisées dans $[0, 1)$, chacun des 2^{32k} vecteurs possibles apparaît le même nombre de fois sur une période complète (à l'exception du vecteur nul).*

Aucun LCG ne peut dépasser une k-distribution de dimension 5 avec des modules raisonnables.

Tempering : justification mathématique

La récurrence linéaire sur GF(2) produit des bits de sortie corrélés de façon prévisible. Le **tempering** est une transformation bijective qui améliore la distribution empirique sans altérer la période. Il s'agit de la composition de quatre opérations :

$$y \leftarrow y \oplus \big((y \gg 11) \,\&\, \texttt{0xFFFFFFFF}\big) \tag{2.3}$$

$$y \leftarrow y \oplus \big((y \ll 7) \,\&\, \texttt{0x9D2C5680}\big) \tag{2.4}$$

$$y \leftarrow y \oplus \big((y \ll 15) \,\&\, \texttt{0xEFC60000}\big) \tag{2.5}$$

$$y \leftarrow y \oplus (y \gg 18) \tag{2.6}$$

Chacune est une **transformation linéaire sur** $\mathrm{GF}(2)^{32}$, donc inversible (matrice triangulaire avec des 1 sur la diagonale). La bijectivité garantit que l'inversion est possible bit par bit (cf. section 4.2).

2.2.2 Justification du code

```python
W, N, M, R = 32, 624, 397, 31
A = 0x9908B0DF    # Constante de twist (matrice A sur GF(2))
F = 1812433253    # Constante d'initialisation (empirique)
U, D = 11, 0xFFFFFFFF
S, B = 7,  0x9D2C5680
T, C = 15, 0xEFC60000
L = 18

def init(seed):
    state = [seed & 0xFFFFFFFF]  # X_0 = seed tronque a 32 bits
    for i in range(1, N):        # 624 iterations
        prev = state[i - 1]
        # X_i = F * (X_{i-1} XOR (X_{i-1} >> 30)) + i
        state.append((F * (prev ^ (prev >> 30)) + i) & 0xFFFFFFFF)
```

```
    return state
```

Listing 2.2 – Initialisation MT19937

Analyse :

- `prev ^ (prev » 30)` : XOR de `prev` avec lui-même décalé de 30 bits, mélangeant bits forts et faibles pour éviter des états initiaux trop proches pour des graines similaires.
- `+ i` : l'indice différencie chaque position.
- $F = 1812433253$ a été choisi expérimentalement pour sa bonne diffusion lors de l'initialisation.

```
def twist(state):
    for i in range(N):
        bit_haut = state[i] & 0x80000000        # Bit 31 de state[i]
        bits_bas = state[(i + 1) % N] & 0x7FFFFFFF  # Bits 0-30 de state[i+1]
        x = bit_haut | bits_bas                  # Concatenation
        x_shifted = x >> 1
        if x & 1:                                # Si LSB = 1 : XOR avec A
            x_shifted ^= A
        state[i] = state[(i + M) % N] ^ x_shifted  # M = 397
    return state
```

Listing 2.3 – Twist MT19937

Analyse :

- `0x80000000` et `0x7FFFFFFF` : masques des bits fort/faible implémentant la concaténation $(x_k^u \,|\, x_{k+1}^l)$.
- `state[(i + M) % N]` : saut de $M = 397$ introduisant la dépendance non locale qui assure la longue période.
- Le XOR conditionnel avec A implémente la multiplication par la matrice de twist sur GF(2).

```
def temper(y):
    y ^= (y >> U) & D     # Etape (5.2)
    y ^= (y << S) & B     # Etape (5.3)
    y ^= (y << T) & C     # Etape (5.4)
    y ^= y >> L           # Etape (5.5)
    return y & 0xFFFFFFFF
```

```python

def generate(seed, n):
    state = init(seed)
    index = N            # Forcer un twist immediat a la premiere extraction
    results = []
    for _ in range(n):
        if index >= N:  # Etat epuise : regenerer
            twist(state)
            index = 0
        results.append(temper(state[index]))
        index += 1
    return results
```

Listing 2.4 – Tempering et génération MT19937

Analyse : `index = N` au départ force le twist dès la première extraction. Le twist est déclenché *exactement* toutes les 624 sorties, ce qui rend le MT rapide (un twist pour 624 valeurs). L'ordre et les constantes des étapes de tempering ont été optimisés pour maximiser le score aux tests statistiques de distribution (TestU01).

2.2.3 Interprétation et application

Le MT19937 est le générateur par défaut de Python (`random`), C++ (`std::mt19937`), R, Ruby, PHP et NumPy. Ses propriétés statistiques en font le choix standard pour la **simulation scientifique**.

Limite fondamentale

Après 624 observations, l'état interne est entièrement reconstruit (cf. section 4.2). Le MT19937 ne doit jamais être utilisé en cryptographie.

2.3 Transformée de Box-Muller

2.3.1 Théorie mathématique

La transformée de Box-Muller (1958) est une méthode exacte pour convertir des variables uniformes en variables gaussiennes par **changement de variables**.

Théorème 2.3.1 (Box-Muller, 1958)**.** *Soient* $U_1, U_2 \sim \mathcal{U}(0,1)$ *indépendantes. Définis-*

sons :

$$Z_0 = \sqrt{-2\ln U_1} \cdot \cos(2\pi U_2) \tag{2.7}$$
$$Z_1 = \sqrt{-2\ln U_1} \cdot \sin(2\pi U_2) \tag{2.8}$$

Alors Z_0 et Z_1 sont indépendantes et suivent toutes deux la loi $\mathcal{N}(0,1)$.

Démonstration. Posons $R = \sqrt{-2\ln U_1}$ et $\Theta = 2\pi U_2$. La transformation inverse est :

$$U_1 = e^{-(Z_0^2+Z_1^2)/2}, \qquad U_2 = \frac{\arctan(Z_1/Z_0)}{2\pi}$$

Le Jacobien de la transformation $(U_1, U_2) \to (Z_0, Z_1)$ vaut :

$$|J| = \left|\frac{\partial(u_1, u_2)}{\partial(z_0, z_1)}\right| = \frac{1}{2\pi}\, e^{-(z_0^2+z_1^2)/2}$$

Comme $f_{U_1,U_2}(u_1, u_2) = 1$ (densité uniforme conjointe), la densité conjointe de (Z_0, Z_1) est :

$$f_{Z_0,Z_1}(z_0, z_1) = f_{U_1,U_2}(u_1, u_2) \cdot |J|^{-1} = \frac{1}{\sqrt{2\pi}}\, e^{-z_0^2/2} \cdot \frac{1}{\sqrt{2\pi}}\, e^{-z_1^2/2}$$

qui est exactement le produit de deux densités $\mathcal{N}(0,1)$: Z_0 et Z_1 sont donc indépendantes et standard normales. □

Interprétation géométrique et loi de Rayleigh

Si $Z_0, Z_1 \sim \mathcal{N}(0,1)$ indépendantes, alors $R = \sqrt{Z_0^2 + Z_1^2}$ suit une **loi de Rayleigh** de densité :

$$f_R(r) = r\, e^{-r^2/2}, \quad r \geq 0$$

La CDF est $F_R(r) = 1 - e^{-r^2/2}$. En inversant :

$$F_R(r) = U_1 \implies r = \sqrt{-2\ln(1-U_1)} \approx \sqrt{-2\ln U_1}$$

le remplacement de $1 - U_1$ par U_1 étant valide car les deux ont la même distribution uniforme.

2.3.2 Justification du code

```python
def box_muller(u1, u2):
    if u1 <= 0:
        u1 = 1e-10    # Garde : evite ln(0) = -inf
    if u2 <= 0:
        u2 = 1e-10
```

```
    r     = math.sqrt(-2.0 * math.log(u1))  # Rayon de Rayleigh
    theta = 2.0 * math.pi * u2              # Angle uniforme sur
        [0, 2*pi)
    z0 = r * math.cos(theta)                # Premiere gaussienne
    z1 = r * math.sin(theta)                # Deuxieme gaussienne
    return z0, z1

def box_muller_series(uniform_rng, seed, n):
    n_uniform = ((n + 1) // 2) * 2     # Plus petit entier pair >= n
    uniforms  = uniform_rng(seed, n_uniform)
    results   = []
    for i in range(0, len(uniforms) - 1, 2):
        z0, z1 = box_muller(uniforms[i], uniforms[i + 1])
        results.extend([z0, z1])
    return results[:n]
```

Listing 2.5 – Transformée de Box-Muller

Analyse :

— Les gardes `u1 <= 0` protègent contre `log(0) = -inf` (valeurs limites en virgule flottante).

— `((n+1)//2)*2` : calcule le plus petit entier pair $\geq n$, car Box-Muller consomme des paires (U_1, U_2).

— `results[:n]` : tronque au nombre exact demandé (le dernier Z_1 peut être en excès si n est impair).

— La qualité des sorties dépend entièrement du générateur `uniform_rng` : si celui-ci présente des corrélations, la preuve ci-dessus ne s'applique plus.

Pour générer des variables $\mathcal{N}(\mu, \sigma^2)$, il suffit d'appliquer $X = \mu + \sigma Z$ où $Z \sim \mathcal{N}(0, 1)$.

2.4 Blum-Blum-Shub (BBS)

2.4.1 Théorie mathématique

Le BBS (Blum, Blum & Shub, 1986) est le premier PRNG prouvé **cryptographiquement sûr** sous une hypothèse de complexité.

Définition 2.4.1 (Nombre de Blum)**.** *$M = p \cdot q$ est un* nombre de Blum *si p et q sont deux nombres premiers distincts tels que $p \equiv q \equiv 3 \pmod 4$.*

Définition 2.4.2 (Problème de résiduosité quadratique – QRP)**.** *Étant donné un nombre de Blum M et un entier a avec symbole de Jacobi $(a/M) = 1$, déterminer si a est un résidu quadratique modulo M est un problème supposé difficile en temps polynomial.*

L'algorithme génère les termes de la suite :

$$x_{n+1} = x_n^2 \bmod M \tag{2.9}$$

et extrait le(s) bit(s) de poids faible de chaque x_{n+1}.

Théorème 2.4.1 (Sécurité de BBS – Blum, Blum, Shub, 1986)**.** *Si QRP est difficile, alors la séquence de bits produits par le LSB de BBS est polynomialement indistinguable d'une séquence vraiment aléatoire : aucun algorithme en temps polynomial ne peut prédire le prochain bit avec probabilité supérieure à* $\frac{1}{2} + \frac{1}{\mathrm{poly}(|M|)}$.

Sketch de réduction. Supposons un algorithme $\mathcal{A}$ qui prédit le bit suivant avec probabilité $\frac{1}{2} + \varepsilon$. On construit un algorithme $\mathcal{B}$ qui résout QRP avec probabilité liée à ε : $\mathcal{B}$ simule $\mathcal{A}$ sur la séquence BBS et utilise sa réponse pour distinguer les résidus quadratiques des pseudo-carrés. Si QRP est difficile, ε doit être négligeable. □

Condition $p \equiv q \equiv 3 \pmod 4$

Cette condition garantit que la suite est **purement périodique** : la racine carrée de tout résidu quadratique modulo M est elle-même un résidu quadratique, ce qui permet de remonter la séquence dans les deux sens et assure que $f(x) = x^2 \bmod M$ est bijective sur l'ensemble des résidus quadratiques.

2.4.2 Justification du code

```python
# Petits premiers de Blum pour tests pedagogiques
SMALL_PRIMES = {'p': 499, 'q': 547}
# Verification : 499 = 4*124 + 3 => 499 mod 4 = 3 (OK)
#                547 = 4*136 + 3 => 547 mod 4 = 3 (OK)

def bbs(seed, p, q, n):
    M = p * q                       # Module de Blum : M = p*q
    x = (seed * seed) % M           # Prechauffage : x_1 = seed^2 mod M
    results = []
    for _ in range(n):
        x = (x * x) % M             # Recurrence : x_{k+1} = x_k^2 mod M
        results.append(x & 1)       # Extraction du LSB (seul bit provablement sur)
    return results
```

Listing 2.6 – Générateur Blum-Blum-Shub

Analyse :

— `x = (seed*seed) % M` : la pré-itération place la graine dans l'ensemble des résidus quadratiques de $\mathbb{Z}_M^*$, condition nécessaire pour la preuve de sécurité.

— `x = (x*x) % M` : squaring modulaire. En pratique crypto, on utiliserait `pow(x, 2, M)` (optimisé). Python gère nativement les grands entiers – pas d'overflow.

— `x & 1` : extraction du LSB. C'est le seul bit prouvablement sûr par itération ; on peut extraire jusqu'à $\lfloor \log_2 \log_2 M \rfloor$ bits, soit environ 10 bits pour $|M| = 1024$ bits.

— Les paramètres $p = 499, q = 547$ sont **uniquement pédagogiques**. En production, p et q doivent avoir au moins 512 bits chacun.

2.5 Hash_DRBG (NIST SP 800-90A)

2.5.1 Théorie mathématique

Le Hash_DRBG est un *Deterministic Random Bit Generator* standardisé par le NIST (SP 800-90A Rev. 1, 2015), certifié FIPS 140-2. Il est basé sur SHA-256 et maintient un état interne $(\mathbf{V}, \mathbf{C}, \mathit{reseed_counter})$ où $\mathbf{V}$ et $\mathbf{C}$ sont des vecteurs de $\mathit{seedlen} = 440\,\text{bits} = 55\,\text{octets}$.

Fonction de dérivation Hash_df

$$\text{Hash_df}(\mathit{input}, \ell) = \Big\|_i = 1^{\lceil \ell/32 \rceil}\text{SHA256}\big(i \,\|\, \ell \,\|\, \mathit{input}\big)\big[\,:\ell\big]$$

Le préfixe i (compteur sur 1 octet) garantit que les blocs sont distincts même si *input* est identique.

Instanciation

$$\mathit{seed_material} = \mathit{entropy} \,\|\, \mathit{nonce} \,\|\, \mathit{personalization} \tag{2.10}$$

$$\mathit{seed} = \text{Hash_df}(\mathit{seed_material},\ \mathit{seedlen}) \tag{2.11}$$

$$\mathbf{V} = \mathit{seed} \tag{2.12}$$

$$\mathbf{C} = \text{Hash_df}(\texttt{0x00} \,\|\, \mathit{seed},\ \mathit{seedlen}) \tag{2.13}$$

Le préfixe `0x00` assure la *domain separation* entre les calculs de $\mathbf{V}$ et de $\mathbf{C}$ (convention NIST).

Génération

$$W = \|_i = 0^{m-1}\text{SHA256}(\mathbf{V} + i) \tag{2.14}$$

$$output = W[: num_bytes] \tag{2.15}$$

$$H = \text{SHA256}(\texttt{0x03} \,\|\, \mathbf{V}) \tag{2.16}$$

$$\mathbf{V}_{\text{new}} = (\mathbf{V} + H + \mathbf{C} + reseed_counter) \bmod 2^{seedlen \times 8} \tag{2.17}$$

La mise à jour de **V** assure la **backtracking resistance** : SHA-256 étant une fonction à sens unique, connaître **V** au temps t ne permet pas de retrouver les sorties aux temps antérieurs.

Théorème 2.5.1 (Sécurité du Hash_DRBG – informel)**.** *Sous l'hypothèse que SHA-256 se comporte comme une fonction aléatoire (modèle de l'oracle aléatoire), les sorties du Hash_DRBG sont indistinguables d'une séquence vraiment aléatoire par tout adversaire en temps polynomial.*

2.5.2 Justification du code

```python
SEED_LEN = 55    # 440 bits = seedlen pour Hash_DRBG avec SHA-256

def _hash_df(input_data, num_bytes):
    hash_len   = 32    # SHA-256 produit 32 octets
    # ceil(num_bytes / 32)
    num_blocks = (num_bytes + hash_len - 1) // hash_len
    result     = b""
    for counter in range(1, num_blocks + 1):
        # counter (1 octet) || num_bytes (4 octets) || input_data
        to_hash = (counter.to_bytes(1, "big")
                   + num_bytes.to_bytes(4, "big")
                   + input_data)
        result += _sha256(to_hash)
    return result[:num_bytes]    # Tronquer au nombre exact
```

Listing 2.7 – Fonction de dérivation Hash_df

```python
def drbg_instantiate(entropy=None, nonce=None, personalization=b"")
    :
    if entropy is None:
        entropy = os.urandom(SEED_LEN)
    if nonce is None:
```

```python
        nonce = os.urandom(SEED_LEN // 2)
    seed = _hash_df(entropy + nonce + personalization, SEED_LEN)
    V    = seed
    C    = _hash_df(b"\x00" + seed, SEED_LEN)  # 0x00 = domain separation
    return {"V": V, "C": C, "reseed_counter": 1}

def drbg_generate(state, num_bytes):
    m    = (num_bytes + 32 - 1) // 32
    W    = b""
    data = state["V"]
    for _ in range(m):
        W   += _sha256(data)
        # Incrementer V comme entier big-endian
        int_data = (int.from_bytes(data, "big") + 1) % (2**(len(data)*8))
        data = int_data.to_bytes(len(state["V"]), "big")
    output = W[:num_bytes]
    # Mise a jour : backtracking resistance
    H     = _sha256(b"\x03" + state["V"])  # 0x03 = domain separation
    mod   = 2 ** (SEED_LEN * 8)
    new_v = (int.from_bytes(state["V"], "big")
             + int.from_bytes(H, "big")
             + int.from_bytes(state["C"], "big")
             + state["reseed_counter"]) % mod
    state["V"]             = new_v.to_bytes(SEED_LEN, "big")
    state["reseed_counter"] += 1
    return output, state

def drbg_reseed(state, entropy=None):
    if entropy is None:
        entropy = os.urandom(SEED_LEN)
    # 0x01 = domain separation pour reseed
    seed          = _hash_df(b"\x01" + state["V"] + entropy, SEED_LEN)
    state["V"]    = seed
    state["C"]    = _hash_df(b"\x00" + seed, SEED_LEN)
    state["reseed_counter"] = 1
    return state
```

Listing 2.8 – Instanciation et génération du DRBG

Analyse :

— Les préfixes `0x00`, `0x01`, `0x03` assurent la *domain separation* entre les différentes opérations, conformément au NIST SP 800-90A.

— La formule $\mathbf{V}_{\text{new}} = (\mathbf{V} + H + \mathbf{C} + \textit{reseed_counter}) \bmod 2^{440}$ mélange quatre quantités distinctes pour maximiser la diffusion.

— Le reseed incorpore la nouvelle entropie avec l'état courant, assurant qu'un attaquant connaissant l'ancien état ne peut prédire les sorties futures après reseed.

2.6 os.urandom

2.6.1 Théorie : sources d'entropie physique

`os.urandom` n'est pas un algorithme : c'est une interface vers l'entropie physique du système d'exploitation, dont la sécurité repose sur l'imprévisibilité fondamentale de phénomènes physiques.

Architecture Linux

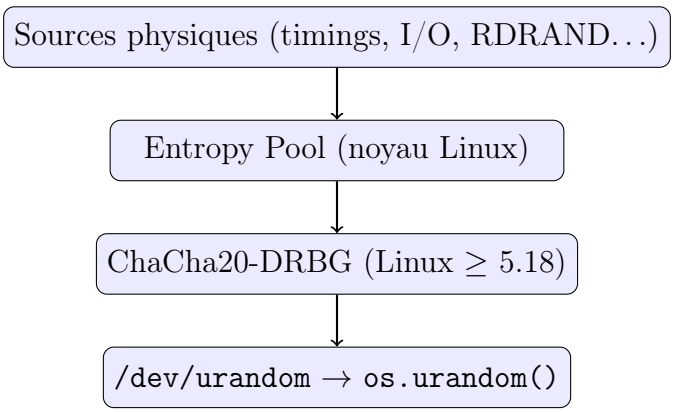

FIGURE 2.1 – Architecture de `os.urandom` sous Linux

Définition 2.6.1 (Min-entropie)**.** *La min-entropie d'une source X est :*

$$H_\infty(X) = -\log_2\Big(\max_x P(X = x)\Big)$$

C'est la borne inférieure garantie de l'entropie, indépendante de la distribution complète. Le noyau accumule des contributions jusqu'à atteindre un seuil de ≈ 128 bits de min-entropie.

Depuis Linux 5.6, la distinction entre `/dev/random` (bloquant) et `/dev/urandom` (non-bloquant) n'existe plus. `os.urandom` utilise `getrandom(GRND_BLOCK)` (Linux 3.17+), qui bloque uniquement au démarrage jusqu'à l'initialisation du pool.

2.6.2 Justification du code

```python
def os_generate_bytes(n):
    return os.urandom(n)    # Syscall direct : getrandom() (Linux) ou
                            # CryptGenRandom() (Windows)

def os_next_int32():
    # Interprete 4 octets aleatoires comme entier big-endian 32 bits
    return int.from_bytes(os.urandom(4), "big")

def os_next_float():
    # Mappe [0, 2^32 - 1] vers [0, 1 - 2^{-32})
    return os_next_int32() / (2**32)
```

Listing 2.9 – Interface os.urandom

Analyse :

— `os.urandom(n)` est un simple wrapper syscall ; aucune logique applicative n'est nécessaire.

— La normalisation par 2^{32} (et non $2^{32}-1$) mappe dans $[0,1)$ (borne supérieure exclue), ce qui est préférable pour des algorithmes comme Box-Muller qui nécessitent $U \in (0,1)$.

2.7 XOR NRBG (Non-Deterministic Random Bit Generator)

2.7.1 Théorie mathématique

Théorème 2.7.1 (Préservation d'entropie par XOR)**.** *Si A et B sont deux variables aléatoires indépendantes, et si au moins l'une d'elles est uniformément distribuée sur $\{0,1\}$, alors $A \oplus B$ est uniformément distribuée.*

Démonstration. Supposons $A \sim \mathcal{U}(\{0,1\})$ et B quelconque, indépendante de A.

$$\begin{aligned}
P(A \oplus B = 0) &= P(A=0, B=0) + P(A=1, B=1) \\
&= P(A=0) \cdot P(B=0) + P(A=1) \cdot P(B=1) \quad \text{(indépendance)} \\
&= \tfrac{1}{2} P(B=0) + \tfrac{1}{2} P(B=1) \quad \text{(A uniforme)} \\
&= \tfrac{1}{2} \left(P(B=0) + P(B=1)\right) = \tfrac{1}{2}
\end{aligned}$$

De même $P(A \oplus B = 1) = \frac{1}{2}$, donc $A \oplus B$ est uniforme, **indépendamment des propriétés de** B. □

Corollaire 2.7.1 (Robustesse aux sources compromises)**.** *Pour k sources indépendantes $X_1, \ldots, X_k$ de min-entropies $H_1, \ldots, H_k$:*

$$H_\infty(X_1 \oplus \cdots \oplus X_k) \geq \max(H_1, \ldots, H_k)$$

Si une seule source est sûre (haute entropie), la sortie combinée l'est aussi.

Limitation : si toutes les sources sont *corrélées* ou *compromises simultanément*, la propriété ne tient plus.

2.7.2 Justification du code

```python
def xor_combine_bits(sources):
    """XOR bit-a-bit de plusieurs sources."""
    if not sources: return []
    n = len(sources[0])
    result = []
    for i in range(n):
        bit = 0             # Element neutre : 0 XOR x = x
        for source in sources:
            bit ^= source[i]    # XOR cumulatif
        result.append(bit)
    return result

def xor_combine_bytes(sources):
    """XOR octet par octet (= 8 XOR de bits paralleles)."""
    if not sources: return b''
    n      = len(sources[0])
    result = bytearray(n)    # Initialise a 0x00 (element neutre)
    for i in range(n):
        xor_val = 0
        for source in sources:
            xor_val ^= source[i]
        result[i] = xor_val
    return bytes(result)

def xor_nrbg(generators, seeds, n):
    """Generateur hybride : combine plusieurs PRNGs via XOR."""
    if len(generators) != len(seeds):
```

```
        raise ValueError("Nombre generateurs != nombre graines")
    # Evaluer chaque generateur independamment
    outputs = [gen(seed, n) for gen, seed in zip(generators, seeds)
        ]
    result  = []
    for i in range(n):
        xor_val = 0
        for output in outputs:
            xor_val ^= output[i]    # XOR des sorties en position i
        result.append(xor_val)
    return result
```

Listing 2.10 – XOR NRBG

Analyse :

— `bit = 0` : l'élément neutre du XOR garantit $0 \oplus x = x$ lors de l'accumulation.

— `[gen(seed, n) for gen, seed in zip(generators, seeds)]` : évalue tous les générateurs indépendamment en une expression.

— La vérification `len(generators) != len(seeds)` prévient une erreur silencieuse si un générateur était omis.

Chapitre 3

Tests statistiques

Les tests évaluent des séquences d'octets $\in [0, 255]$. Le niveau de signification est $\alpha = 0{,}05$: un générateur parfait échouerait en moyenne à 5 % des tests par chance.

3.1 Entropie de Shannon

3.1.1 Théorie mathématique

Définition 3.1.1 (Entropie de Shannon)**.** *Pour une variable aléatoire discrète X prenant les valeurs $\{x_1, \ldots, x_k\}$ avec probabilités $\{p_1, \ldots, p_k\}$:*

$$H(X) = -\sum_{i=1}^{k} p_i \log_2 p_i \quad \textit{(en bits)}$$

avec la convention $0 \log_2 0 = 0$.

Théorème 3.1.1 (Maximum d'entropie)**.** *$H(X) \leq \log_2 k$, avec égalité si et seulement si X est uniformément distribuée.*

Démonstration. Par l'inégalité de Jensen (log est concave) :

$$H(X) = \mathbb{E}[-\log_2 p(X)] \leq -\log_2 \mathbb{E}[p(X)] = -\log_2 \left(\sum_i p_i^2 \right) \leq -\log_2 \left(\frac{1}{k} \right) = \log_2 k$$

L'égalité dans Jensen est atteinte ssi $p(X)$ est constante, i.e. $p_i = 1/k$ pour tout i. □

Application aux octets

Pour $k = 256$ valeurs possibles :

$$H_{\max} = \log_2 256 = 8 \text{ bits/octet}$$

Le seuil de passage est $H > 7{,}9$ bits/octet (98,75 % du maximum).

Biais de l'estimateur

Pour n octets i.i.d. uniformes, le biais de l'estimateur de Shannon est :

$$\mathbb{E}[\hat{H}] \approx H - \frac{k-1}{2n} + O(n^{-2})$$

Pour $n = 10\,000$: biais $\approx 255/20\,000 \approx 0{,}013$ bits – négligeable.

3.1.2 Justification du code

```python
def shannon_entropy(data):
    freq    = Counter(data)    # {valeur : nombre_occurrences}
    n       = len(data)
    entropy = 0.0
    for count in freq.values():
        p = count / n          # Estimateur MLE : p_i = count_i / n
        if p > 0:              # Protege contre log_2(0)
            entropy -= p * math.log2(p)  # Contribution : -p_i *
                log_2(p_i)
    return entropy
```

Listing 3.1 – Entropie de Shannon

Analyse :

- `Counter(data)` : dict `{valeur: occurrences}` en $O(n)$.
- Les valeurs absentes ne contribuent pas (probabilité estimée 0, contribution $0 \log_2 0 = 0$).
- La garde `if p > 0` évite `nan` en virgule flottante.

Limitation : l'entropie de Shannon ne détecte pas les corrélations entre symboles consécutifs – seulement la distribution marginale. Un générateur produisant $0, 1, 2, \ldots, 255, 0, 1, \ldots$ en séquence ordonnée aurait $H = 8$ mais serait trivialement prévisible.

3.2 Test du Chi-carré

3.2.1 Théorie mathématique

Hypothèses :

H_0 : les données suivent une distribution uniforme sur $\{0, \ldots, 255\}$

H_1 : les données ne suivent pas cette distribution

Pour $k = 256$ classes, fréquence attendue $E = n/256$, la statistique est :

$$\chi^2 = \sum_{i=0}^{255} \frac{(O_i - E)^2}{E} \tag{3.1}$$

Théorème 3.2.1 (Pearson, 1900)**.** *Sous H_0 (et si $E \geq 5$), la statistique χ^2 de l'équation (3.1) converge en loi vers une distribution χ^2 à $\mathrm{df} = k - 1 = 255$ degrés de liberté quand $n \to \infty$.*

Sketch. Chaque terme $(O_i - E)/\sqrt{E}$ converge vers une $\mathcal{N}(0, 1)$ par le TCL. La somme des carrés de 256 normales standard donne une $\chi^2(256)$. La contrainte $\sum O_i = n$ retire 1 degré de liberté, donnant $\mathrm{df} = 255$. □

Pour $\alpha = 0{,}05$ et $\mathrm{df} = 255$:

$$\chi^2_{\text{critique}} = 293{,}25 \quad (\text{quantile } 0{,}95 \text{ de } \chi^2(255))$$

On rejette H_0 si $\chi^2 > 293{,}25$. Note : $\mathbb{E}[\chi^2] = 255$ et $\mathrm{Var}[\chi^2] = 510$ sous H_0 ; la valeur critique est à environ 2,1 écarts-types au-dessus de la moyenne.

3.2.2 Justification du code

```python
def chi_squared_test(data, p=0.05):
    freq     = Counter(data)
    n        = len(data)
    expected = n / 256          # E = n/256 pour chaque valeur
    chi2     = 0.0
    for i in range(256):        # Toutes les valeurs possibles [0, 255]
        observed = freq.get(i, 0)    # O_i = 0 si valeur absente
        chi2 += ((observed - expected) ** 2) / expected
    critical_value = 293.25     # Quantile 0.95 de chi2(255) -- hardcode
    return {
        'chi2'            : chi2,
        'degrees_freedom': 255,
        'critical_value' : critical_value,
        'status'          : 'PASS' if chi2 < critical_value else 'FAIL'
    }
```

Listing 3.2 – Test du Chi-carré

Analyse : `range(256)` itère sur *toutes* les valeurs, y compris les absentes : une valeur manquante contribue $(0-E)^2/E = E$ à la statistique, ce qui est correct. La valeur critique est codée en dur pour éviter une dépendance à `scipy`.

3.3 Autocorrélation

3.3.1 Théorie mathématique

Définition 3.3.1 (Coefficient d'autocorrélation)**.** *Pour une séquence* $(X_1, \ldots, X_n)$ *et un décalage* k *:*

$$r(k) = \frac{\mathrm{Cov}(X_i, X_{i+k})}{\mathrm{Var}(X)} = \frac{\displaystyle\sum_{i=1}^{n-k}(X_i - \bar{X})(X_{i+k} - \bar{X})}{\displaystyle\sum_{i=1}^{n}(X_i - \bar{X})^2}$$

Propriété 3.3.1. *Pour un signal i.i.d. de taille* n *:*

$$r(k) \sim \mathcal{N}\left(0, \frac{1}{n}\right), \quad k \neq 0$$

L'intervalle de confiance à 95 % est $\left[-\frac{1{,}96}{\sqrt{n}}, \frac{1{,}96}{\sqrt{n}}\right]$*. Pour* $n = 10\,000$ *:* $\pm 0{,}0196$*.*

3.3.2 Justification du code

```python
def autocorrelation(data, lag=1):
    n        = len(data) - lag          # Nombre de paires disponibles
    moy      = sum(data) / len(data)    # Moyenne empirique
    variance = sum((x - moy)**2 for x in data)
    if variance == 0:
        return 0.0   # Donnees constantes : non defini -> 0 par convention
    covariance = sum(
        (data[i] - moy) * (data[i + lag] - moy) for i in range(n)
    )
    return covariance / variance   # Normalisation par variance totale
```

Listing 3.3 – Autocorrélation

Analyse :

— La normalisation par la variance totale (et non la variance des paires) garantit $r(0) = 1$ par construction.

— La fonction de test évalue les décalages $[1, 8, 16, 32]$: lag $= 1$ détecte les corrélations consécutives (les plus critiques pour le LCG), les lags plus grands détectent des périodicités plus longues.

3.4 Test de Kolmogorov-Smirnov (KS)

3.4.1 Théorie mathématique

Hypothèses :

H_0 : les données suivent la distribution uniforme sur $[0, 255]$

H_1 : les données ne suivent pas cette distribution

Définition 3.4.1 (Statistique KS)**.** *Soit $F_n(x)$ la fonction de répartition empirique (FRE) :*

$$F_n(x) = \frac{\#\{i : X_i \leq x\}}{n}$$

La statistique de Kolmogorov-Smirnov est :

$$D_n = \sup_x \big|F_n(x) - F(x)\big|$$

où $F(x) = x/255$ est la CDF de la loi uniforme sur $[0, 255]$.

Théorème 3.4.1 (Kolmogorov, 1933)**.**

$$P\big(\sqrt{n}\, D_n > t\big) \xrightarrow{n\to\infty} 2\sum_{k=1}^{\infty}(-1)^{k+1}e^{-2k^2t^2}$$

Pour $\alpha = 0{,}05$: $t^* = 1{,}36$, soit la valeur critique asymptotique :

$$D_{\text{critique}} = \frac{1{,}36}{\sqrt{n}}$$

Théorème 3.4.2 (Glivenko-Cantelli)**.** *Si les observations sont i.i.d. selon F :*

$$\sup_x |F_n(x) - F(x)| \xrightarrow{n\to\infty} 0 \quad p.s.$$

Donc sous H_0, $D_n \to 0$ et $\sqrt{n}\, D_n$ converge en loi vers la distribution de Kolmogorov.

3.4.2 Justification du code

```python
def kolmogorov_smirnov_test(data):
    normalized = sorted([x / 255.0 for x in data])  # Normaliser +
        trier
```

```
    n           = len(normalized)
    max_diff    = 0.0
    for i, value in enumerate(normalized):
        empi = (i + 1) / n    # F_n(x) = rang / n (apres tri)
        theo = value           # F(x) = x/255 (loi uniforme
           normalisee)
        diff = abs(empi - theo)
        max_diff = max(max_diff, diff)
    critical_value = 1.36 / math.sqrt(n)
    return {
        'D'              : max_diff,
        'critical_value': critical_value,
        'n'              : n,
        'status'         : 'PASS' if max_diff < critical_value else
           'FAIL'
    }
```

Listing 3.4 – Test de Kolmogorov-Smirnov

Analyse :

— Après tri, le rang $i+1$ correspond directement à F_n : exactement $i+1$ valeurs sont $\leq$ à la $(i+1)$-ième valeur triée.

— `max_diff = max(max_diff, diff)` : calcul du supremum sur toutes les valeurs – définition exacte de D_n.

— Le test KS est sensible aussi bien aux décalages de distribution qu'aux déformations de forme, contrairement au χ^2 qui ignore l'ordre des valeurs.

Chapitre 4

Attaques

4.1 Récupération de la graine LCG

4.1.1 Modèle de menace

L'attaquant observe quelques sorties consécutives $X_1, X_2, \ldots$ du LCG et connaît les paramètres publics (a, c, m). Son objectif est de retrouver X_0 pour prédire toutes les sorties futures et passées.

4.1.2 Méthode 1 : Résolution algébrique

Théorème 4.1.1 (Inversion du LCG)**.** *Sachant $X_1 = (a \cdot X_0 + c) \bmod m$, la graine se récupère en une opération :*

$$X_0 = a^{-1} \cdot (X_1 - c) \bmod m$$

où a^{-1} est l'inverse modulaire de a modulo m, calculé via l'algorithme d'Euclide étendu en $O(\log m)$.

L'inverse modulaire existe si et seulement si $\gcd(a, m) = 1$, condition satisfaite pour tous les paramètres standards (glibc, MMIX). La prédiction de toutes les sorties futures est alors parfaite à 100 %.

```python
def recover_seed_algebrique(x1, x2, x3, a, c, m):
    a_inv = pow(a, -1, m)              # Inverse modulaire (Python 3.8+)
    x0    = ((x1 - c) * a_inv) % m     # X_0 = a^{-1} * (X_1 - c) mod m
    # x2, x3 servent a la validation
    return x0
```

Listing 4.1 – Méthode algébrique – récupération de graine LCG

Analyse : `pow(a, -1, m)` est la syntaxe Python 3.8+ pour l'inverse modulaire (Euclide étendu, $O(\log m)$). La soustraction `x1 - c` peut être négative ; l'opérateur `%` ramène correctement dans $[0, m-1]$ en Python.

4.1.3 Méthode 2 : Force brute

```python
def recover_seed_bruteforce(outputs, a, c, m, seed_max=1000000):
    for candidate in range(seed_max):
        generated = lcg(candidate, a, c, m, len(outputs))
        if generated == outputs:    # Validation sur toute la sequence
            return candidate
    return None
```

Listing 4.2 – Méthode force brute – récupération de graine LCG

Analyse : Pour une graine tirée d'un timestamp Unix (précision à la seconde), l'espace est $\approx$ 86 400 valeurs par jour – attaquable en moins d'une seconde. La validation sur `len(outputs)` valeurs réduit les faux positifs à probabilité $m^{-\texttt{len(outputs)}}$ négligeable.

4.1.4 Méthode 3 : Known-plaintext (chiffrement XOR)

Si le LCG est utilisé comme générateur de flux pour un chiffrement XOR ($C = P \oplus$ LCG_keystream) et que l'attaquant connaît un couple (P, C) :

```python
def recover_seed_from_xor(plaintext, ciphertext, a, c, m, seed_max=100000):
    keystream      = xor_bytes(plaintext, ciphertext)  # K = P XOR C
    keystream_ints = list(keystream)
    for candidate in range(seed_max):
        generated       = lcg(candidate, a, c, m, len(keystream))
        generated_bytes = [x % 256 for x in generated]  # Reduction a 1 octet
        if generated_bytes == keystream_ints:
            return candidate
    return None
```

Listing 4.3 – Attaque known-plaintext sur chiffrement XOR LCG

Analyse : `x % 256` réduit chaque sortie LCG à un octet. `K = P XOR C` : si l'attaquant connaît un couple (clair, chiffré), il récupère directement le keystream. Cette attaque illustre pourquoi utiliser un PRNG non-crypto comme générateur de flux est catastrophique.

TABLE 4.1 – Comparaison des trois méthodes d'attaque LCG

Méthode	**Complexité**	**Condition requise**
Algébrique	$O(\log m)$	1 sortie observée
Force brute	$O(seed_max)$	Espace de graines restreint
Known-plaintext	$O(seed_max)$	1 couple (clair, chiffré)

4.2 Reconstruction d'état du Mersenne Twister

4.2.1 Modèle de menace

L'attaquant observe **624 sorties consécutives de 32 bits** du MT19937 et veut reconstruire l'état interne complet ($state[0], \ldots, state[623]$) pour prédire toutes les sorties futures. L'état interne est un vecteur de $N = 624$ mots de 32 bits ; chaque sortie révèle (via tempering inversible) un mot d'état. Il faut exactement N observations.

4.2.2 Inversibilité du tempering

Le tempering est une bijection linéaire sur $\mathrm{GF}(2)^{32}$. Chaque opération est inversible :

Inversion de $y \oplus= (y \gg \textbf{\textit{shift}})$

Les *shift* bits de poids fort de y' sont identiques à ceux de y (le décalage droite les remplace par des 0, donc XOR ne les modifie pas). Pour $i \geq shift$: $\texttt{bit}_i(y) = \texttt{bit}_i(y') \oplus \texttt{bit}_{i+shift}(y)$, déjà connu car reconstruit de gauche à droite.

```python
def invert_right_shift_xor(y, shift):
    result = 0
    for i in range(32):
        bit_pos  = 31 - i          # De poids fort vers poids faible
        if i < shift:
            bit = (y >> bit_pos) & 1            # Bits hauts inchanges
        else:
            prev_bit    = (result >> (bit_pos + shift)) & 1
            current_bit = (y >> bit_pos) & 1
            bit = current_bit ^ prev_bit        # Inversion
        result |= (bit << bit_pos)
    return result
```

Listing 4.4 – Inversion décalage droite XOR

Inversion de $y \oplus= (y \ll \textbf{\textit{shift}})$ & $\textbf{\textit{mask}}$

S'inverse de droite à gauche (poids faible vers poids fort) car le décalage gauche amène des 0 en position basse.

```
def invert_left_shift_xor_mask(y, shift, mask):
    result = 0
    for i in range(32):              # De poids faible vers poids fort
        if i < shift:
            bit = (y >> i) & 1                    # Bits bas inchanges
        else:
            prev_bit    = (result >> (i - shift)) & 1
            mask_bit    = (mask >> i) & 1
            current_bit = (y >> i) & 1
            bit = current_bit ^ (prev_bit & mask_bit)
        result |= (bit << i)
    return result
```

Listing 4.5 – Inversion décalage gauche XOR masqué

Fonction untemper complète

```
def untemper(y):
    # Inverser dans l'ordre INVERSE du tempering (composition de fonctions)
    y = invert_right_shift_xor(y, L)          # Inverse etape (5.5) : L=18
    y = invert_left_shift_xor_mask(y, T, C)   # Inverse etape (5.4) : T=15
    y = invert_left_shift_xor_mask(y, S, B)   # Inverse etape (5.3) : S=7
    y = invert_right_shift_xor(y, U)          # Inverse etape (5.2) : U=11
    return y
```

Listing 4.6 – Inversion complète du tempering

Principe : si $\mathcal{T} = T_4 \circ T_3 \circ T_2 \circ T_1$, alors $\mathcal{T}^{-1} = T_1^{-1} \circ T_2^{-1} \circ T_3^{-1} \circ T_4^{-1}$.

4.2.3 Reconstruction et prédiction

```python
def recover_state(outputs):
    if len(outputs) < N:
        raise ValueError(f"Besoin de {N} sorties, recu {len(outputs
            )}")
    return [untemper(outputs[i]) for i in range(N)]

def predict_next(state, index, n):
    state       = state.copy()    # Ne pas modifier l'etat original
    predictions = []
    for _ in range(n):
        if index >= N:
            state = twist(state)
            index = 0
        predictions.append(temper(state[index]))
        index += 1
    return predictions
```

Listing 4.7 – Reconstruction d'état et prédiction MT19937

Théorème 4.2.1 (Correction de l'attaque MT)**.** *Si* `outputs[0..623]` *sont les 624 sorties consécutives d'un MT19937, alors* `recover_state(outputs)` *reconstruit exactement l'état interne, et* `predict_next` *prédit toutes les sorties futures avec une précision de* 100 %.

Démonstration. (1) `untemper(temper(x)) = x` pour tout x de 32 bits : chaque opération de tempering est bijective sur $\mathrm{GF}(2)^{32}$, et `untemper` applique les inverses dans l'ordre inverse.
(2) Les 624 sorties consécutives proviennent exactement de $state[i_0], \ldots, state[i_0 + 623]$. La propriété (1) garantit la reconstruction exacte.
(3) Après reconstruction, `predict_next` simule exactement le comportement futur du MT car il utilise les mêmes fonctions `twist` et `temper`. □

Complexité totale : $O(624)$ inversions + $O(n)$ prédictions, exécutable en moins d'une milliseconde.

Chapitre 5

Recommandations et analyse critique

5.1 Tests statistiques $\neq$ sécurité cryptographique

C'est la leçon centrale de ce projet. Les tests de Shannon, chi-carré, autocorrélation et KS évaluent l'**apparence aléatoire** d'une distribution marginale. Ils sont aveugles à la structure algébrique d'un générateur.

Table 5.1 – Résultats des tests vs résistance aux attaques

Générateur	**Shannon**	χ^2	**Autocorr.**	**KS**	**Résistance attaque**
LCG (glibc)	PASS	PASS	Léger écart	PASS	**Nulle** (1 sortie)
MT19937	PASS	PASS	PASS	PASS	**Nulle** (624 sorties)
Hash_DRBG	PASS	PASS	PASS	PASS	Élevée (NIST)
os.urandom	PASS	PASS	PASS	PASS	Très élevée

Un attaquant ne cherche pas un biais statistique – il cherche une structure algébrique exploitable. Ces deux propriétés sont **orthogonales**.

5.2 Principe de Kerckhoffs

Formulé en 1883 : *« La sécurité d'un système cryptographique ne doit pas dépendre du secret de l'algorithme, mais uniquement du secret de la clé. »* Appliqué aux générateurs : un bon CSPRNG doit rester imprévisible même si l'algorithme est public et que l'attaquant observe un grand nombre de sorties passées.

5.3 Incidents réels

— **Casinos en ligne (2007–2010)** : sites de poker utilisant MT19937 initialisé avec le timestamp Unix. Espace des graines $\approx$ 86 400 valeurs/jour. Des groupes observaient

les premières cartes, reconstruisaient l'ordre du sabot par force brute et prédisaient toutes les mains adverses.

— **Android SecureRandom (2013)** : implémentation défectueuse réutilisant des graines faibles pour `java.util.Random` dans des portefeuilles Bitcoin. Des attaquants ont récupéré des clés privées.

— **Debian OpenSSL (2008)** : un commit supprimait l'initialisation de l'entropie, réduisant l'espace à $\approx 32\,768$ valeurs (le PID du processus). Toutes les clés SSH/SSL générées entre 2006 et 2008 sur Debian/Ubuntu étaient compromises.

5.4 Tableau comparatif des générateurs

TABLE 5.2 – Synthèse comparée des générateurs

Générateur	Vitesse	Sécurité crypto	Qualité stat.	Usage recommandé
LCG	Très élevée	Nulle	Passable	Simulation, jeux
MT19937	Élevée	Nulle	Excellente	Simulation scient.
Box-Muller	Héritée	Héritée	Variable	Distributions $\mathcal{N}$
BBS	Très faible	Prouvée(QRP)	Bonne	Protocoles formels
Hash_DRBG	Moyenne	Très élevée	Excellente	Crypto, FIPS
os.urandom	Moyenne	Très élevée	Excellente	Clés, tokens, nonces
XOR NRBG	Variable	Dépend srces	Bonne	Haute disponibilité

5.5 Règles fondamentales

1. **Ne jamais utiliser LCG ou MT19937** pour des clés, nonces, IV ou tokens d'authentification.
2. **Préférer `secrets` ou `os.urandom`** (Python) pour tout besoin de sécurité – recommandation officielle Python.
3. **Utiliser Hash_DRBG (NIST SP 800-90A)** pour les applications nécessitant une certification FIPS.
4. **Ne pas initialiser un PRNG avec le timestamp seul** : l'espace des graines est trivial pour un attaquant.
5. **Appliquer le principe de Kerckhoffs** : concevoir le système en supposant que l'algorithme est public.
6. **Reseeder régulièrement** les CSPRNG à longue durée de vie avec de l'entropie fraîche issue de `os.urandom`.
7. **Distinguer simulation et cryptographie** : un excellent PRNG de simulation peut être un désastre cryptographique.

Chapitre 6

Conclusion

Ce projet a démontré une distinction fondamentale : **les propriétés statistiques et la sécurité cryptographique sont indépendantes**.

Le LCG et le MT19937 produisent des séquences statistiquement excellentes – ils passent l'entropie de Shannon, le chi-carré, l'autocorrélation et le test KS avec des scores comparables aux CSPRNG. Pourtant, le LCG est compromis en une opération algébrique à partir d'une seule observation, et le MT19937 est entièrement reconstruit en moins d'une milliseconde après 624 observations.

La preuve mathématique de sécurité du BBS (sous hypothèse QRP) et la certification NIST du Hash_DRBG illustrent ce que signifie réellement "cryptographiquement sûr" : pas seulement une bonne apparence statistique, mais une **réduction formelle** de la capacité de prédiction à un problème computationnellement difficile.

Les incidents réels – casinos en ligne, Android 2013, Debian OpenSSL 2008 – confirment que ces vulnérabilités théoriques ont des conséquences financières et légales directes. La règle invariable : **choisir le bon générateur pour le bon contexte**.

Bibliographie

— **Blum, L., Blum, M., & Shub, M.** (1986). A simple unpredictable pseudo-random number generator. *SIAM Journal on Computing*, 15(2), 364–383.

— **Box, G. E. P., & Muller, M. E.** (1958). A note on the generation of random normal deviates. *The Annals of Mathematical Statistics*, 29(2), 610–611.

— **Johnston, D.** (2018). *Random Number Generators — Principles and Practices*. De Gruyter.

— **Knuth, D. E.** (1997). *The Art of Computer Programming, Vol. 2 : Seminumerical Algorithms* (3rd ed.). Addison-Wesley.

— **Marsaglia, G.** (1968). Random numbers fall mainly in the planes. *PNAS*, 61(1), 25–28.

— **Matsumoto, M., & Nishimura, T.** (1998). Mersenne Twister : A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM TOMACS*, 8(1), 3–30.

— **NIST.** (2015). *SP 800-90A Rev. 1 : Recommendation for Random Number Generation Using Deterministic Random Bit Generators*.

— **Pearson, K.** (1900). On the criterion that a given system of deviations. . .*Philosophical Magazine*, 50(302), 157–175.

— **Shannon, C. E.** (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.

— **Kolmogorov, A. N.** (1933). Sulla determinazione empirica di una legge di distribuzione. *Giornale dell'ISIA*, 4, 83–91.